

Traditional RAG vs. Graph RAG

Everything a technical professional needs to choose, design, and evaluate the right retrieval architecture from core mechanics and failure modes to cost structures, latency trade-offs, and a decision framework for production systems.

~35%

Reported accuracy gain on multi-hop queries with Graph RAG (AWS/Lettra benchmark)

10–100×

Higher indexing cost for Graph RAG vs traditional RAG (Microsoft GraphRAG study)

60%

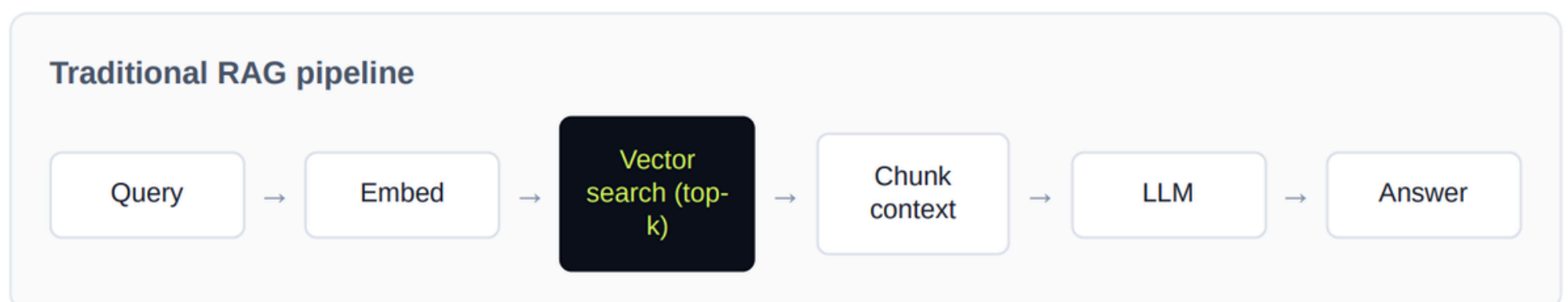
Of AI projects predicted to fail by Gartner due to weak retrieval/data foundations

Two fundamentally different retrieval topologies.

Both systems sit between a user query and an LLM, and both aim to ground the model in external knowledge. But the shape of that knowledge flat vectors versus connected entities changes everything downstream.

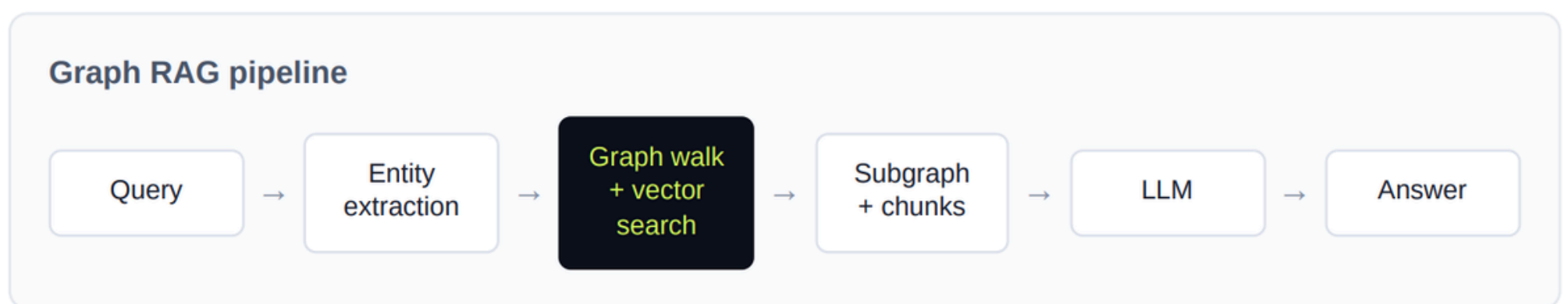
Traditional RAG: the vector-similarity pipeline

A traditional RAG system treats the knowledge base as a bag of chunks. Documents are split into passages (typically 200–1,000 tokens), each chunk is embedded into a dense vector using a model like `textembedding-3-large`, and the vectors are stored in a vector database (Pinecone, Chroma, Weaviate, pgvector). At query time, the user's question is embedded and the top-k nearest neighbors by cosine similarity are retrieved and stuffed into the LLM prompt. It's fast, simple, and cheap to build and it's been the production baseline since 2023.



Graph RAG the knowledge-graph pipeline

Graph RAG treats the knowledge base as a network of entities. During indexing, an LLM (or NER pipeline) extracts triples (subject, predicate, object) from every document. Entities become nodes, relationships become edges, and the resulting knowledge graph is stored in Neo4j, Kùzu, TigerGraph, or similar. At query time, the system does hybrid retrieval: it identifies anchor entities from the query, traverses the graph 1–3 hops to collect connected context, and pulls relevant text chunks then merges both into the LLM prompt.



Why the difference matters

Traditional RAG Mental Model

"Find me text that looks like the question."

- Retrieval = cosine similarity over embeddings
- Each chunk is evaluated independently
- No awareness of how chunks relate to each other
- Answers come from whichever chunks happen to land in the top-k window

Graph RAG mental model

"Find me facts that are connected to the question."

- Retrieval = deterministic graph traversal + vector search
- Entities carry explicit relationships
- Multi-hop paths preserved across documents
- Answers can cite a traceable chain of evidence

Ten dimensions that matter in production.

Teams picking a retrieval architecture typically get seduced by accuracy numbers on a benchmark. Production reality is messier indexing cost, update latency, explainability requirements, and the skill set of the team matter just as much.

Dimension	Traditional RAG	Graph RAG
Retrieval mechanism	Cosine similarity over dense vector embeddings (ANN search)	Graph traversal (Cypher / SPARQL) + vector search as a hybrid step
Data representation	Flat text chunks embedded as n-dimensional vectors	Typed entities (nodes) + typed relationships (edges) with properties
Multi-hop reasoning	Weak — chunks retrieved independently, no logical chaining	Strong — k-hop traversal explicitly follows $A \rightarrow B \rightarrow C$ paths
Cross-document linking	Only if the same entity appears semantically in both chunks	Native — shared entities create explicit edges across sources
Explainability	Low — retrieval is a black-box similarity score	High — retrieval path is a traceable subgraph with named edges
Indexing cost	1× baseline (one embedding pass per chunk)	10–100× baseline (LLM extraction + dedup + graph build)
Update latency	Seconds — embed new chunk, upsert to vector store	Minutes to hours — re-extract entities, merge into graph, re-embed
Query latency	~20–100 ms typical ANN lookup	~100–500 ms; grows with traversal depth and fan-out
Scalability	Horizontal scaling is straightforward; vectors shard well	Sharding graphs while preserving edges is architecturally harder
Best data type	Unstructured text: docs, wikis, FAQs, support tickets	Entity-rich domains: legal, healthcare, finance, supply chain

The **failure modes** that reveal architecture fit.

Benchmarks are polite. Production isn't. Here are the query patterns where each approach quietly collapses and what actually happens under the hood.

Where traditional RAG breaks

Failure mode 1: Multi-hop questions

The user asks a question whose answer requires chaining two or three facts together. Each individual fact may sit in its own chunk, but no single chunk contains the answer and similarity search has no mechanism to chain them.

Q: "Which companies founded by former PayPal employees now compete with OpenAI?"

Traditional RAG

Retrieves chunks about PayPal alumni and chunks about OpenAI competitors separately. The LLM sees both but can't verify the cross-reference, so it either hedges or hallucinates a confident wrong answer.

Graph RAG

Traverses (Person)-[FOUNDED]→(Company)-[COMPETES_WITH]→(OpenAI) where Person.worked_at = "PayPal" . Returns an enumerable, verifiable set.

Failure mode 2: Disambiguation across documents

The same entity appears with different surface forms across sources. Vector similarity treats each mention as a fresh chunk; it has no persistent identity for "the entity."

Q: "What is Jane Smith's role in ACME's renewable energy projects?"

Traditional RAG

Retrieves three chunks about "Jane S.", "J. Smith", and "Smith, Jane" may not recognize they're the same person, or may include an unrelated Jane Smith from another document.

Graph RAG

Entity resolution during indexing unifies all surface forms into a single node. One traversal returns every relationship tied to that node.

Failure mode 3: Temporal / versioned facts

Policies, org charts, product specs change over time. Similarity search retrieves whatever is most similar regardless of version Graph RAG can attach [valid_from, valid_to] intervals to edges and query point-in-time.

Where Graph RAG breaks

Failure mode 1: Noisy / ambiguous extraction

LLM extraction is only as good as the prompts and schema. Aggressive extraction creates bloated graphs with weak edges; strict schemas miss critical relationships. A messy graph can perform worse than plain vector search.

Failure mode 2: Questions without entity anchors

Summary, mood, and stylistic queries ("What's the overall tone of Q3 feedback?") have no entity anchor to seed a traversal. Graph RAG falls back to vector search adding latency without value.

Failure mode 3: High-velocity corpora

News feeds, chat logs, and real-time telemetry churn faster than graph extraction can keep up. Vector indexes with streaming CDC refresh in seconds; graph re-extraction is typically batch-scheduled.

The hidden economics of retrieval choice.

Accuracy dominates the marketing. Infrastructure bills dominate the quarterly review. Here's a realistic view of what each architecture actually costs to run at enterprise scale.

Indexing cost: the one-time (but recurring) bill

Traditional RAG indexing is essentially "embed each chunk once." At ~\$0.0001 per 1K tokens, indexing a 100K document corpus (avg 2K tokens each) costs around \$20 in API fees plus storage. Graph RAG requires an LLM to extract entities and relationships from every document often multiple passes for NER, relation extraction, and disambiguation.

Microsoft's GraphRAG project documented graph construction costs running 10–100× standard RAG indexing.

The same corpus may cost \$1,000–\$10,000+ to index the first time, and updates are non-trivial because entity resolution is a global operation.

\$20–50

Traditional RAG: one-time indexing cost for ~100K docs

\$1K–10K+

Graph RAG: one-time indexing cost for the same corpus

Ongoing

Graph rebuild is not one-and-done — schemas drift, entities evolve

Query latency: what the user feels

For single-hop questions, traditional RAG runs 20–100 ms for the ANN lookup. Graph RAG adds entity linking plus traversal; depending on depth ($k=1$ vs $k=3$) and fan-out, expect 100–500 ms for retrieval alone. On deep traversals over large graphs, worst-case latency can exceed a second before the LLM even starts generating.

Scalability: what happens at 100M documents

Vector databases shard with simple consistent hashing. Graph databases don't sharding a graph while preserving edge locality is an active research problem, and cross-shard traversals degrade performance significantly. Most production Graph RAG caps at 10–100M nodes on one beefy instance, or uses distributed graph engines like TigerGraph or Neptune Analytics.

Total cost of ownership: the honest view

Traditional RAG TCO

- **Infra:** Vector DB, \$0.10–0.30/M vectors/month managed
- **Embeddings:** Pennies for build, pennies for queries
- **Ops:** ~0.25 FTE to maintain pipelines
- **Hidden cost:** Accuracy ceiling on complex queries

Graph RAG TCO

- **Infra:** Graph DB + vector DB, ~3–5× traditional RAG
- **LLM extraction:** Dominant cost during index + refresh
- **Ops:** 0.5–1 FTE for schema and extraction tuning
- **Hidden cost:** Ontology drift as the domain evolves